

PARCS-Agent: Enabling AI Agents to Exploit Distributed Parallel Computing via the Model Context Protocol

Oleksii Bohusevych ¹ 

¹ Faculty of Computer Science and Cybernetics, Taras Shevchenko National University of Kyiv, 01601 Kyiv, Ukraine; oleksii.bohusevych@knu.ua

* Correspondence: oleksii.bohusevych@knu.ua

1 Abstract

Large language model (LLM) agents have demonstrated impressive capabilities in tool use, code generation, and scientific reasoning, yet they remain bound by the throughput of a single execution thread. Computationally intensive tasks—Monte Carlo simulation, combinatorial optimisation, hyperparameter search, large-scale graph analysis—either time out or require severe undersampling when handled sequentially. We present PARCS-Agent, a system that exposes a cloud-scale distributed computing cluster to an AI agent through three Model Context Protocol (MCP) tools: `get_cluster_info`, `create_session`, and `run_layer`. The agent submits arbitrary C# computation logic; the platform compiles it with Roslyn at runtime, fans it out across up to 21 Kubernetes worker pods via Google Cloud Pub/Sub and KEDA autoscaling, and returns aggregated results. We introduce the PARCS-Agent Benchmark, a suite of 15 tasks spanning quantitative finance, bioinformatics, epidemiology, machine learning, cryptography, and atmospheric science, designed so that the sequential wall-clock cost substantially exceeds practical agent time budgets. Experiments show that PARCS-enabled agents achieve mean speedups of 15–18× over sequential baselines while delivering statistically equivalent or superior solution quality. PARCS-Agent is, to our knowledge, the first system to use MCP as the interface between an AI agent and a live distributed HPC cluster, and the first benchmark to measure *speedup ratio* as a first-class agent capability metric.

Keywords: AI agents; Model Context Protocol; distributed computing; parallel computation; LLM tool use; benchmarking; Kubernetes; high-performance computing

1. Introduction

2. Related Work

2.1. Tool-Augmented Language Model Agents

The capacity of large language models (LLMs) to interact with external tools has been a central theme in recent agent research. Schick et al. [?] demonstrated in Toolformer that models can learn, in a self-supervised manner, when and how to call external APIs—including a calculator, a search engine, and a calendar—augmenting their reasoning with structured information that lies beyond the model’s parametric knowledge. Yao et al. [?] proposed ReAct, which interleaves chain-of-thought reasoning with environment-facing actions such as web searches and database lookups, enabling iterative problem-solving in which each observation informs subsequent reasoning steps. These foundational works established the agent–tool paradigm that PARCS-Agent extends to distributed computation.

Subsequent work has scaled tool use to larger collections of APIs. ToolLLM [?] curates 16,000 real-world REST APIs into a benchmark (ToolBench) and trains a model to select and invoke them with high accuracy using a novel depth-first search-based decision tree. Gorilla [?] focuses on

machine-learning framework APIs (HuggingFace, TorchHub, TensorFlow Hub) and demonstrates that retrieval-augmented generation substantially reduces hallucinated API calls. In the scientific domain, SciAgent [?] assembles a corpus of over 30,000 tool-augmented samples spanning mathematics, physics, chemistry, and finance, and introduces SciToolBench for evaluation across five scientific disciplines. A comprehensive survey by Mialon et al. [?] taxonomises augmented language models along the dimensions of memory, reasoning, and action, identifying computation as an underexplored tool class.

All of these systems treat tools as stateless function calls that return data or compute a result locally and synchronously. PARCS-Agent departs from this model: its `run_layer` tool submits C# code to a live Kubernetes cluster, fans the computation out across up to 21 parallel workers via a Pub/Sub queue, and collects results asynchronously—a qualitatively different interaction semantics that scales execution effort with the number of available workers.

LLMCompiler [?] is the closest prior work in its use of parallelism. It decomposes a user query into a directed acyclic graph (DAG) of tool calls and dispatches independent branches concurrently, reducing end-to-end latency by up to $3.7\times$ on multi-hop question-answering benchmarks. However, LLMCompiler parallelises *existing* heterogeneous tool calls (e.g., concurrent web searches); it does not compile new code, provision worker pods, or fan out a uniform computation across a homogeneous pool. CodeAct [?] proposes Python as the universal action space for agents, maintaining a stateful interpreter that accumulates results across interaction turns. CodeAct executes code locally and sequentially; PARCS-Agent compiles C# via the Roslyn compiler at session time and distributes the resulting binary to remote pods, enabling true data-parallel computation at cluster scale.

2.2. LLMs for High-Performance and Parallel Computing

A parallel line of inquiry asks whether LLMs can *generate* parallel code targeting established HPC programming models. Nichols et al. [?] introduce ParEval, a benchmark of 420 problems drawn from numerical computing, graph algorithms, and Monte Carlo methods, and evaluate generated code in MPI, OpenMP, CUDA, and Kokkos. They find that state-of-the-art models perform significantly worse on parallel code than on equivalent serial implementations, and that correctness rates decline sharply as algorithmic complexity increases—a finding that directly motivates the PARCS-Agent architecture. HPC-Coder-V2 [?] addresses this gap by fine-tuning models on HPC-INSTRUCT, a synthetic corpus of 120,000 parallel code samples, achieving improved parallel code generation, particularly for MPI and OpenMP. A complementary evaluation benchmarks recent frontier models on classical HPC kernels (conjugate gradient, heat equation, DGEMM, and others) and reports that even highly capable models struggle with cache-oblivious algorithms and non-trivial loop transformations [?].

PARCS-Agent takes a structurally different approach. Rather than asking the agent to write the parallel infrastructure—thread management, communication primitives, synchronisation barriers—PARCS provides these as platform services. The agent writes only the *per-worker business logic*, expressed as a single C# class implementing a minimal two-method interface. The platform handles fan-out scheduling, pod isolation, serialisation of inputs and outputs, and result aggregation. This architecture makes parallel computation accessible to an agent without HPC expertise, bypassing the parallel code generation bottleneck identified by [?].

He et al. [?] apply LLMs to a different HPC challenge: automated optimisation of the task-to-processor mapper in the Legion distributed runtime framework. Their system uses the Legion mapper DSL as an intermediate representation and an LLM to propose mapping strategies, reducing what previously required days of expert tuning to minutes of automated search. Unlike PARCS-Agent, this work presupposes an existing parallelised application whose mapping requires optimisation; it does not address the problem of an agent devising and executing a novel parallel computation from scratch.

2.3. Agent Benchmarks for Computational Tasks

The community has invested considerable effort in constructing benchmarks that stress-test agent capabilities across complex, multi-step tasks. GAIA [?] evaluates general-purpose AI assistants on

466 real-world questions requiring web navigation, file manipulation, and multi-modal reasoning; top models achieve approximately 75% on Level 1 questions, while Level 3 questions remain largely unsolved. SWE-bench [?] targets software engineering specifically, presenting 2,294 real GitHub issues and requiring agents to produce patches that pass the associated test suites, with a current state-of-the-art solve rate of approximately 50% for the verified subset. ScienceAgentBench [?] narrows the focus to data-driven scientific discovery, curating 102 tasks from 44 published papers across bioinformatics, computational chemistry, geographical information science, and psychology; the best evaluated agent solves approximately 34% of tasks autonomously. TheAgentCompany [?] benchmarks agents on realistic workplace tasks spanning software engineering, data analysis, and project management in a simulated company environment.

None of these benchmarks include a parallelism or computational-scale dimension. Tasks are designed to be completed by a single agent process within a reasonable wall-clock budget, and speedup is not a measured quantity. The PARCS-Agent Benchmark is designed along an orthogonal axis: each of the 15 tasks requires computational effort that substantially exceeds what a single-threaded agent can deliver within a practical time budget (≥ 15 min sequential). The primary evaluation metric is therefore the *speedup ratio*—wall-clock time of a PARCS-enabled agent relative to a sequential baseline—alongside solution quality. To our knowledge, speedup ratio has not previously been proposed as a first-class agent evaluation metric.

2.4. The Model Context Protocol and Agent Tool Ecosystems

The Model Context Protocol (MCP), introduced by Anthropic in 2024, defines a standard JSON-RPC-over-SSE interface for exposing tools, resources, and prompts to LLM agents, enabling interoperability across model providers and tool implementors [?]. Hou et al. [?] survey the growing MCP ecosystem and identify security concerns including prompt injection, tool poisoning, and privilege escalation, but find no existing work on using MCP to expose distributed HPC infrastructure. A complementary analysis of published MCP tool descriptions identifies substantial quality deficiencies—over half of surveyed tools lack adequate purpose descriptions [?]—motivating careful tool description design. These findings informed the design of the PARCS-Agent MCP tool descriptions, which include explicit parameter contracts, example values, and constraint documentation (parallelism must not exceed `maxParallelism` returned by `get_cluster_info`).

PARCS-Agent is, to our knowledge, the first system to expose a live cloud-scale parallel computing cluster as a set of MCP tools, and the first to evaluate AI agent performance on tasks where access to such tools is the decisive factor separating a complete, accurate solution from an incomplete or timed-out one.

3. System Architecture

4. PARCS-Agent Benchmark

5. Experiments

6. Discussion

7. Conclusions

Author Contributions: For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used “Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.”, please turn to the [CRediT taxonomy](#) for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding: Please add: “This research received no external funding” or “This research was funded by NAME OF FUNDER grant number XXX.” and and “The APC was funded by XXX”. Check carefully that the details given

are accurate and use the standard spelling of funding agency names at <https://search.crossref.org/funding>, any errors may affect your future funding.

Institutional Review Board Statement: In this section, you should add the Institutional Review Board Statement and approval number, if relevant to your study. You might choose to exclude this statement if the study did not require ethical approval. Please note that the Editorial Office might ask you for further information. Please add “The study was conducted in accordance with the Declaration of Helsinki, and approved by the Institutional Review Board (or Ethics Committee) of NAME OF INSTITUTE (protocol code XXX and date of approval).” for studies involving humans. OR “The animal study protocol was approved by the Institutional Review Board (or Ethics Committee) of NAME OF INSTITUTE (protocol code XXX and date of approval).” for studies involving animals. OR “Ethical review and approval were waived for this study due to REASON (please provide a detailed justification).” OR “Not applicable” for studies not involving humans or animals.

Informed Consent Statement: Any research article describing a study involving humans should contain this statement. Please add “Informed consent was obtained from all subjects involved in the study.” OR “Patient consent was waived due to REASON (please provide a detailed justification).” OR “Not applicable” for studies not involving humans. You might also choose to exclude this statement if the study did not involve humans.

Written informed consent for publication must be obtained from participating patients who can be identified (including by the patients themselves). Please state “Written informed consent has been obtained from the patient(s) to publish this paper” if applicable.

Data Availability Statement: We encourage all authors of articles published in MDPI journals to share their research data. In this section, please provide details regarding where data supporting reported results can be found, including links to publicly archived datasets analyzed or generated during the study. Where no new data were created, or where data is unavailable due to privacy or ethical restrictions, a statement is still required. Suggested Data Availability Statements are available in section “MDPI Research Data Policies” at <https://www.mdpi.com/ethics>.

Acknowledgments: In this section you can acknowledge any support given which is not covered by the author contribution or funding sections. This may include administrative and technical support, or donations in kind (e.g., materials used for experiments).

Conflicts of Interest: Declare conflicts of interest or state “The authors declare no conflicts of interest.” Authors must identify and declare any personal circumstances or interest that may be perceived as inappropriately influencing the representation or interpretation of reported research results. Any role of the funders in the design of the study; in the collection, analyses or interpretation of data; in the writing of the manuscript; or in the decision to publish the results must be declared in this section. If there is no role, please state “The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results”.

Abbreviations

The following abbreviations are used in this manuscript:

MDPI	Multidisciplinary Digital Publishing Institute
DOAJ	Directory of open access journals
TLA	Three letter acronym
LD	Linear dichroism

Appendix A

Appendix A.1

The appendix is an optional section that can contain details and data supplemental to the main text—for example, explanations of experimental details that would disrupt the flow of the main text but nonetheless remain crucial to understanding and reproducing the research shown; figures of replicates for experiments of which representative data are shown in the main text can be added here if brief, or as Supplementary Data. Mathematical proofs of results not central to the paper can be added as an appendix.

Table A1. This is a table caption.

Title 1	Title 2	Title 3
Entry 1	Data	Data
Entry 2	Data	Data

Appendix B

All appendix sections must be cited in the main text. In the appendices, Figures, Tables, etc. should be labeled, starting with “A”—e.g., Figure A1, Figure A2, etc.

References

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*; Curran Associates: Red Hook, NY, USA, 2023.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*; OpenReview.net, 2023.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.

Patil, S.G.; Zhang, T.; Wang, X.; Gonzalez, J.E. Gorilla: Large Language Model Connected with Massive APIs. *arXiv* **2023**, arXiv:2305.15334.

Ma, Z.; Gou, Z.; Hao, J.; Xu, K.; Hu, Z.; Yin, C.; Zheng, H.; Huang, M.; Deng, Y. SciAgent: Tool-Augmented Language Models for Scientific Reasoning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024.

Mialon, G.; Dessì, R.; Lomeli, M.; Nalmpantis, C.; Pasunuru, R.; Raileanu, R.; Rozière, B.; Schick, T.; Dwivedi-Yu, J.; Celikyilmaz, A.; et al. Augmented Language Models: A Survey. *Trans. Mach. Learn. Res.* **2023**.

Kim, S.; Moon, S.; Tabrizi, R.; Lee, N.; Mahoney, M.W.; Keutzer, K.; Gholami, A. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*; PMLR: Honolulu, HI, USA, 2024; Vol. 235, pp. 24167–24190.

Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; Ji, H. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*; PMLR: Honolulu, HI, USA, 2024; Vol. 235, pp. 50908–50933.

Nichols, D.; Davis, J.H.; Xie, Z.; Rajaram, A.; Bhatele, A. Can Large Language Models Write Parallel Code? In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC 2024)*; ACM: New York, NY, USA, 2024; pp. 117–130.

Nichols, D.; Davis, J.H.; Bhatele, A. HPC-Coder-V2: Studying Code LLMs Across Low-Resource Parallel Languages. *arXiv* **2024**, arXiv:2412.15178.

Xu, C.; Li, M.; Xu, X.; Qin, X.; Chen, P. LLM & HPC: Benchmarking DeepSeek’s Performance in High-Performance Computing Tasks. In *Proceedings of ISC High Performance 2025*; Springer: Cham, Switzerland, 2025.

He, W.; Shen, Z.; Nie, A.; Luo, C.; Nus, T.; Aiken, A.; Zaharia, M. Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*; PMLR, 2025.

Mialon, G.; Fourrier, C.; Swift, C.; Wolf, T.; LeCun, Y.; Scialom, T. GAIA: A Benchmark for General AI Assistants. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.

Jiménez, C.E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; Narasimhan, K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.

- Chen, Z.; Shi, Z.; Wang, S.; et al. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*; OpenReview.net, 2025.
- Xu, F.F.; Ye, Y.; Jain, A.; Du, Y.; et al. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv* **2024**, arXiv:2412.14161.
- Anthropic. Model Context Protocol. Available online: <https://modelcontextprotocol.io> (accessed on 1 May 2026).
- Hou, X.; Zhao, Y.; Wang, S.; Wang, H. Model Context Protocol (MCP): Landscape, Security Threats and Future Research Directions. *arXiv* **2025**, arXiv:2503.23278.
- Luo, L.; et al. Model Context Protocol (MCP) Tool Descriptions Are Smelly! *arXiv* **2025**, arXiv:2602.14878.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.